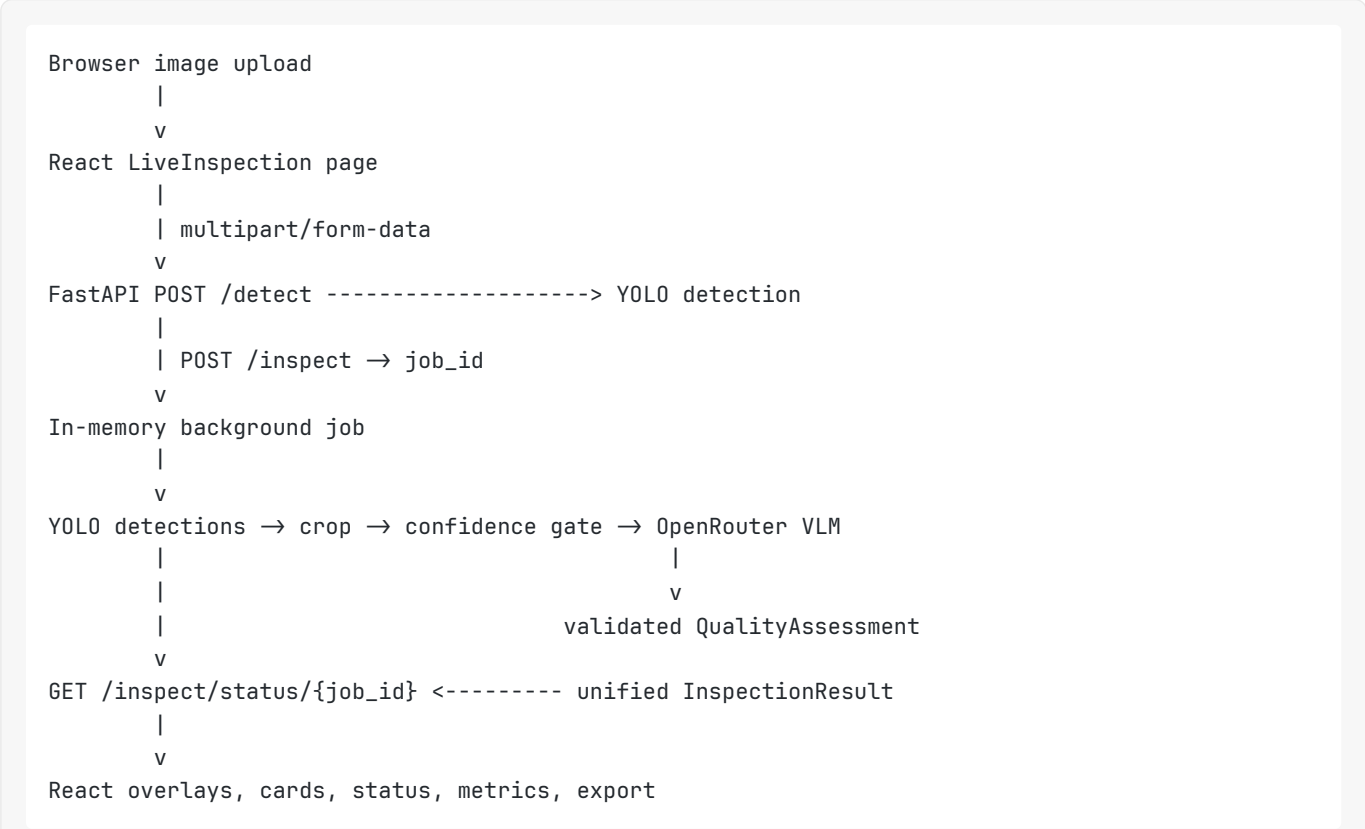
Food Inspection AI

Food Inspection AI is a two-stage computer-vision system for detecting food ingredients and assessing their visible quality. The repository combines a YOLO-based object detector, optional vision-language reasoning, a FastAPI service, a React/TypeScript dashboard, training artifacts, and saved inspection outputs.

The documentation below describes the **implemented repository**, not an idealized future architecture. Where a dataset split, class list, or metric is not directly reproducible from the files committed here, that limitation is stated explicitly.

System at a glance

The system has two operational paths. The detection-only path is designed for low-latency localization. The full inspection path first detects objects, crops each detected object, selects ingredient-specific quality dimensions, sends eligible crops to the configured VLM adapter, validates the returned JSON, and exposes a unified result to the dashboard.



What is implemented

Capability	Implementation	Current status
Ingredient localization	Ultralytics YOLO model loaded from `models/best.pt`	Implemented
Detection API	`POST /detect`	Implemented
Full quality inspection	`POST /inspect` plus status polling	Implemented
Structured quality reasoning	Ingredient profiles plus VLM JSON parsing	Implemented
Browser upload	Drag-and-drop/file selection in `LiveInspection.tsx`	Implemented
Live webcam/video CLI	`backend/live_inference.py`	Implemented
Full local quality CLI	`backend/main.py`	Implemented
Dashboard reports/statistics	React screens and client helpers	Present, but some data is mocked/static
Persistent job storage	None; jobs live in a process-local dictionary	Not implemented
Dataset manifest and split metadata	Dataset YAML is referenced externally from the training run	Not committed

Repository organization

```
.
├── backend/
│   ├── api.py                  # FastAPI endpoints and background job lifecycle
│   ├── inspection_pipeline.py  # YOLO → crop → optional VLM orchestration
│   ├── live_inference.py       # Detection-only webcam/video/image CLI
│   ├── main.py                 # Full local webcam quality-inspection CLI
│   ├── quality_profiles.py     # Ingredient-specific VLM metrics
│   ├── schemas.py              # Shared Pydantic response contract
│   └── vlm_reasoning.py       # VLM interface, adapters, parsing, fallback
├── frontend/
│   ├── client/src/api/         # Axios and inspection request functions
│   ├── client/src/components/  # Layout, inspection cards, overlays, UI primitives
│   ├── client/src/hooks/       # React state orchestration
│   ├── client/src/pages/       # Dashboard, live inspection, reports, model views
│   ├── client/src/types/       # TypeScript mirror of backend contracts
│   └── package.json             # Frontend scripts and dependencies
├── models/
│   └── best.pt                 # Runtime detector weights
├── training/
│   ├── notebooks/              # Training notebook
│   └── runs/                   # Ultralytics run configuration, plots, metrics, weights
├── runtime_artifacts/
│   └── outputs/                # Saved snapshots and JSONL sessions
├── docs/
│   ├── QUALITY_ASSESSMENT.md   # Quality-assessment contract and profiles
│   ├── Final_Report.md         # Timeout diagnosis and async-job history
│   ├── diagnosis.md            # Earlier timeout analysis
│   └── assets/architecture.png # Architecture visual
├── requirements.txt
└── README.md
```

All existing artifacts were retained during restructuring. The spelling of the former `trainning_runs` directory was corrected by moving it to `training/runs`; no training weights, predictions, plots, snapshots, logs, or notebooks were removed.

Machine-learning training record

The committed Ultralytics run is `training/runs/train4`. Its configuration records a pretrained YOLOv9c model, 30 epochs, batch size 8, 640px input images, automatic optimizer selection, an initial learning rate of `0.001`, deterministic seed `0`, validation enabled with `split: val`, and eight workers. The training configuration references `/kaggle/input/newdata3/data (5).yaml`; that dataset YAML and the raw images are not committed, so the exact source manifest and split cardinalities cannot be independently reconstructed from this repository alone.

The configuration records the following augmentation and loss-related settings. These are **training configuration values**, not claims that every transform was separately ablated.

Group	Recorded values
Input and schedule	`imgsz=640`, `epochs=30`, `batch=8`, `optimizer=auto`, `lr0=0.001`, `lrf=0.01`
Reproducibility	`seed=0`, `deterministic=true`, `pretrained=true`
Geometric/color augmentation	`hsv_h=0.015`, `hsv_s=0.7`, `hsv_v=0.4`, `translate=0.1`, `scale=0.5`, `fliplr=0.5`
Composite augmentation	`mosaic=1.0`, `auto_augment=randaugment`, `erasing=0.4`
YOLO loss weights	`box=7.5`, `cls=0.5`, `dfl=1.5`
Validation/NMS-related settings	`val=true`, `split=val`, `iou=0.7`, `max_det=300`

The loss columns in `training/runs/train4/results.csv` are `train/box_loss`, `train/cls_loss`, `train/dfl_loss` and their validation counterparts. The run finished with training losses of `0.94468`, `0.82967`, and `1.02246`, and validation losses of `0.98956`, `1.29326`, and `1.06861`, respectively.

Recorded detection metrics

The final epoch is not automatically the best epoch for every metric. The table therefore separates the final recorded values from the best values found in the 30-row CSV.

Metric	Epoch 1	Final epoch 30	Best recorded value
Precision (B)	0.41639	0.34298	0.45554 at epoch 2
Recall (B)	0.11586	0.25250	0.26704 at epoch 26
mAP@50 (B)	0.08660	0.23750	0.23750 at epoch 30
mAP@50–95 (B)	0.06087	0.17418	0.17418 at epoch 30

The metrics above are the values recorded by Ultralytics in `results.csv`; they should not be interpreted as a complete production benchmark. The repository does not contain the dataset YAML, a locked evaluation protocol, hardware-independent latency benchmark, or class-by-class validation table.

The training artifacts include `results.png`, precision/recall/F1/PR curves, normalized and unnormalized confusion matrices, label visualizations, train batches, validation labels/predictions, `predictions.json`, and `weights/best.pt` plus `weights/last.pt`. The runtime application uses the separate copy at `models/best.pt`.

Split and stratification status

The run records `split: val`, which indicates that Ultralytics evaluated the validation split. It does **not** prove that the split was stratified. No committed manifest states the train/validation/test counts, the class distribution in each split, whether the split was predefined, or whether duplicate images were removed across splits. The README therefore avoids claiming stratification or a particular split percentage.

Detection and quality contracts

`backend/schemas.py` is the central contract shared by the API and the frontend. `Detection` stores the label, integer class ID, confidence, absolute `bbox_xyxy` coordinates, and normalized coordinates. `QualityAssessment` stores the VLM status, optional overall score, metric dictionary, defect labels, explanation, required action, backend name, and measured

latency. `InspectionItem` combines one detection with one quality result. `InspectionResult` adds frame ID, timestamp, source, image size, detection count, and item list.

The status enum is `ok`, `defect`, `uncertain`, or `skipped`. `skipped` is used when the VLM is disabled, when a detection is below the confidence gate, or when a crop cannot be processed. A failed or unparsable VLM response degrades to `uncertain` and requests `flag_for_review`; it does not crash the entire frame pipeline.

A representative result is shaped like this:


```
{
  "frame_id": 17,
  "timestamp": "2026-08-13T10:00:00",
  "source": "upload.jpg",
  "image_size": {"width": 1280, "height": 720},
  "num_detections": 1,
  "items": [
    {
      "detection": {
        "label": "banana",
        "class_id": 4,
        "confidence": 0.91,
        "bbox_xyxy": [120.0, 80.0, 420.0, 500.0],
        "bbox_normalized": [0.09375, 0.11111, 0.32812, 0.69444]
      },
      "quality": {
        "status": "defect",
        "overall_quality_score": 0.45,
        "quality_metrics": {"browning": 0.70, "bruising": 0.50},
        "defects": ["browning", "bruising"],
        "explanation": "Visible browning and bruising are present.",
        "required_action": "remove",
        "vlm_backend": "openrouter",
        "latency_ms": 1840.2
      }
    }
  ]
}
```


Ingredient-aware VLM reasoning

`backend/quality_profiles.py` maps supported labels to visual dimensions. Profiles are present for tomato, banana, apple, potato, strawberry, and onion. Unknown labels fall back to `COMMON_METRICS`: ripeness, bruising, mold, discoloration, and freshness. The profile is inserted into the prompt so the VLM evaluates dimensions appropriate to the detected ingredient rather than using one generic checklist.

`backend/vlm_reasoning.py` defines the abstract `VLMBackend` interface. Its `analyze()` method builds the prompt, times the model call, strips common Markdown fences or surrounding text, parses JSON, validates the semantic fields through Pydantic, and records latency. The module contains adapters for local Qwen2.5-VL, GPT-4o, DashScope Qwen, and OpenRouter. **The current factory returns `OpenRouterBackend` for every requested name.** The other adapters remain in the codebase, but the runtime selection is intentionally documented as OpenRouter-only until the factory is changed.

The default OpenRouter model in the adapter is `google/gemini-flash-1.5-8b`. The API path passes `openrouter` explicitly. The local CLI still exposes several historical `--vlm` choices, but the factory behavior takes precedence, so users should configure `OPENROUTER_API_KEY` for the implemented full-quality path.

End-to-end image flow

The browser entry point is `frontend/client/src/pages/LiveInspection.tsx`. A user drops an image or selects a file. The page passes the file and selected mode to `useInspection`, which delegates to `frontend/client/src/api/inspectionApi.ts`.

For detection-only mode, the client sends the image as a `multipart/form-data` field named `file` to `POST /detect`. FastAPI reads the upload, decodes the bytes with OpenCV, loads the cached YOLO model, and calls `backend.inspection_pipeline.run_inspection()` with no VLM backend.

For full inspection mode, the client sends the same multipart field plus `confidence_gate` to `POST /inspect`. The endpoint decodes the image, creates a UUID job, stores `pending` state in the process-local `_jobs` dictionary, and schedules `process_inspection_job` through FastAPI `BackgroundTasks`. The immediate response is `{"job_id": "...", "status": "pending"}`.

The frontend then polls `GET /inspect/status/{job_id}` approximately once per second. While waiting, it presents upload, YOLO, and VLM progress stages. When the background job finishes, the endpoint returns the unified `InspectionResult`; if processing fails, it returns a failed status and error message. This avoids holding the original HTTP request open while external VLM calls execute.

Inside `run_inspection()`, YOLO produces boxes, class IDs, labels, and confidence scores. Coordinates are clamped while cropping and normalized against the original image width and height. A crop is sent to the VLM only when a backend exists and the detector confidence is at least the configured gate. The result is then rendered by the frontend through `BoundingBoxOverlay`, `DetectionCard`, `ConfidenceBar`, `StatusBadge`, and the live inspection result panel.

The frontend API base URL is `VITE_API_BASE_URL`, defaulting to `http://localhost:8000`. The Axios client uses a 120-second timeout for polling requests and normalizes backend error fields into user-facing errors.

Important file-by-file guide

File	Responsibility and important behavior
`backend/api.py`	FastAPI application, CORS, cached YOLO/VLM constructors, upload decoding, `/detect`, `/inspect`, status polling, and `/health`. The job store is in memory and is not durable or shared across workers.
`backend/inspection_pipeline.py`	Framework-neutral orchestration. It converts Ultralytics boxes into Pydantic detections, crops objects, applies the VLM confidence gate, and preserves typed skipped results.
`backend/live_inference.py`	Detection-only CLI for webcam, video, and still images. It annotates frames and writes JSONL logs plus snapshots under `runtime_artifacts/outputs`.
`backend/main.py`	Full local webcam/video application. It runs detection and optional quality reasoning, draws status-colored boxes, writes structured session logs, and saves snapshots.
`backend/quality_profiles.py`	Controlled ingredient-to-metric mapping and fallback metrics for unknown classes.
`backend/schemas.py`	Pydantic models and enums shared by detection, VLM reasoning, API serialization, and frontend expectations.
`backend/vlm_reasoning.py`	Prompt generation, provider adapters, JSON extraction, semantic normalization, fallback assessment, and latency measurement.
`frontend/client/src/api/client.ts`	Axios base URL, polling timeout, development request logging, and normalized API errors.
`frontend/client/src/api/inspectionApi.ts`	Multipart upload, job submission, one-second status polling, mock history/statistics/model helpers,

File	Responsibility and important behavior
	and frontend-facing progress callbacks.
`frontend/client/src/hooks/useInspection.ts`	React state for result, loading, stage, and error; selects detection-only or full inspection mode.
`frontend/client/src/pages/LiveInspection.tsx`	Main real inspection screen: file input, drag/drop, mode selection, progress, image overlay, overall status, and per-item results.
`frontend/client/src/components/inspection/BoundingBoxOverlay.tsx`	Positions normalized detection boxes over the displayed image.
`frontend/client/src/components/inspection/DetectionCard.tsx`	Displays class, confidence, quality score, explanation, defects, metrics, action, backend, and latency.
`frontend/client/src/components/inspection/ConfidenceBar.tsx`	Visual confidence/score indicator.
`frontend/client/src/components/inspection/StatusBadge.tsx`	Visual status mapping for quality states.
`frontend/client/src/pages/Dashboard.tsx`	Dashboard shell and summary presentation. Some summary values are sourced from client helpers rather than a persistent backend.
`frontend/client/src/pages/Reports.tsx`	Search, sort, filter, JSON preview, and export behavior for inspection history; current history data is mock/static.
`frontend/client/src/pages/ModelInfo.tsx`	Model and API presentation page; several displayed model values/classes are static UI content.
`frontend/client/src/types/inspection.ts`	TypeScript mirror of the backend inspection contracts plus reporting/model view types.
`training/notebooks/food_detection.ipynb`	Exploratory/training notebook retained as a source artifact.
`training/runs/train4/`	Evidence from the recorded training run: configuration, CSV metrics, plots, predictions, validation images, and weights.

File	Responsibility and important behavior
`runtime_artifacts/outputs/`	Existing JSONL sessions and saved image/JSON snapshots retained as runtime examples.
`docs/QUALITY_ASSESSMENT.md`	Detailed quality schema and profile extension notes.
`docs/Final_Report.md` and `docs/diagnosis.md`	Historical timeout analysis explaining the move from synchronous inspection to background jobs with polling.

The many files under `frontend/client/src/components/ui/` are reusable generated-style UI primitives. They are intentionally not duplicated in this README individually because they do not contain domain-specific ML or transport logic; the domain-specific inspection components are documented above.

Running the project

Backend installation

Use Python 3.9 or newer and install the project requirements:

```
git clone https://github.com/Achraf-saadali/Food-Inspection.git
cd Food-Inspection
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Set the environment variables required by the selected runtime. Full VLM inspection currently needs `OPENROUTER_API_KEY`. The local Qwen and alternative API adapters remain available as classes but are not selected by the current factory.

Start the FastAPI backend

Run from the repository root:

```
uvicorn backend.api:app --host 0.0.0.0 --port 8000 --reload
```

Useful endpoints are:

Method	Endpoint	Purpose
`GET`	`/health`	Liveness check
`POST`	`/detect`	Multipart image upload and YOLO-only result
`POST`	`/inspect`	Multipart image upload and immediate job ID
`GET`	`/inspect/status/{job_id}`	Poll job status and retrieve completed result

A minimal detection request is:

```
curl -X POST http://localhost:8000/detect \
  -F "file=@path/to/image.jpg"
```

A full inspection request is:

```
curl -X POST "http://localhost:8000/inspect?confidence_gate=0.4" \
  -F "file=@path/to/image.jpg"
```

Start the frontend

```
cd frontend
pnpm install
pnpm dev
```

To use another backend URL, create `frontend/.env.local` with:

```
VITE_API_BASE_URL=http://localhost:8000
```

Run local CLI inference

Detection-only inference:

```
python -m backend.live_inference
python -m backend.live_inference --source path/to/image.jpg
python -m backend.live_inference --source path/to/video.mp4 --conf 0.4
```

Full webcam quality inspection:

```
OPENROUTER_API_KEY=your_key_here python -m backend.main --vlm openrouter --source 0
```

Press `q` or `Esc` to exit. Press `s` to save an annotated image and JSON report. CLI outputs are written to `runtime_artifacts/outputs/`.

Operational limitations and next engineering steps

The process-local `_jobs` dictionary is suitable for a single-process demonstration or development server, but jobs disappear on restart and are not shared between multiple workers. A production deployment should replace it with durable job storage and a worker system, then add job expiration and concurrency limits.

The API currently allows all CORS origins. This is convenient for development but should be restricted to the deployed frontend origin in production. Upload validation should also enforce file size, MIME type, image dimensions, and a safe decoding policy.

The repository contains no committed dataset manifest or test suite for detector quality. To make the training result reproducible, commit a sanitized dataset YAML, split counts, class names, data provenance, and an evaluation script. To make the product metrics trustworthy, replace the mocked frontend reporting helpers with API-backed persistence and add end-to-end tests for upload, polling, schema validation, and failed VLM calls.

References